Longest Balanced Substring

AlgoExpert · Very Hard · Strings

The problem

- Length of the longest balanced-parentheses substring.
- Two counting passes.

Intuition

- Count opens and closes left to right, then right to left, resetting when the balance breaks.

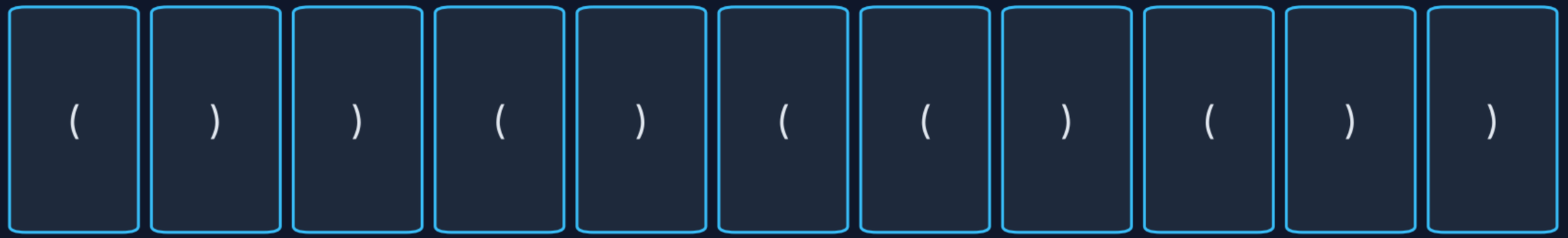
Brute force vs optimal

- Check every substring → $O(n^3)$
- Stack of indices → O(n), O(n) space
- Two counter scans → O(n), O(1)

Approach

- 1. Left to right, equal counts give a candidate
- 2. Reset when closes exceed opens
- 3. Repeat right to left

Worked example



Find the longest balanced substring

Worked example

```
left→right: count opens/closes

equal → candidate; closes > opens → reset
```

Left-to-right scan

Worked example

```
right→left scan catches extra '(' cases

answer: 8
```

Answer: 8

Java solution

```java
public class LongestBalancedSubstring {
    public static int longestBalancedSubstring(String s) {
        return Math.max(scan(s, false), scan(s, true));
    }

    private static int scan(String s, boolean reverse) {
        int open = 0, close = 0, best = 0;
        for (int k = 0; k < s.length(); k++) {
            char c = s.charAt(reverse ? s.length() - 1 - k : k);
            boolean opening = (c == '(') != reverse;
            if (opening) open++; else close++;
            if (open == close) best = Math.max(best, 2 * close);
            else if (close > open) open = close = 0;
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Need both directions
- Reset correctly
- Empty → 0

Interview tips & follow-ups

- Mention the stack approach

Recap

- Count opens and closes left to right, then right to left, resetting when the balance breaks.
- Complexity: O(n)